HTML & CSS

Préparation au métier de développeur

④ Responsive design, grid et flex

PHOEN-IX | ALTHERR MARK

Formateur informatique – Infrastructure
Développement – Interopérabilité

2025-2026

LE RESPONSIVE DESIGN

➜ Définition

Le « Responsive », « Responsive Design » ou encore « Design Adaptatif » est une technique de conception d'interfaces numériques qui fait en sorte que les éléments **s'adaptent en fonction de la taille de l'écran**

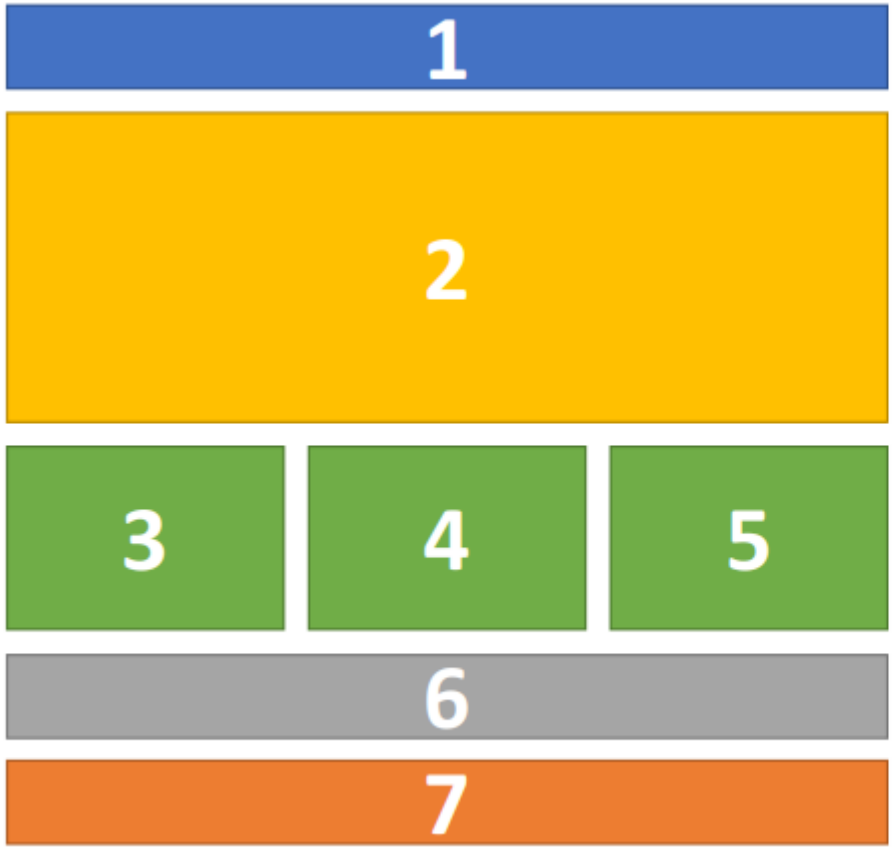
➜ Positionnement des blocs de haut en bas plutôt que cote à cote lorsque la largeur est réduite

➜ Possibilité de considérer que certains blocs moins importants sont inutiles sur mobiles et ainsi les faire disparaître

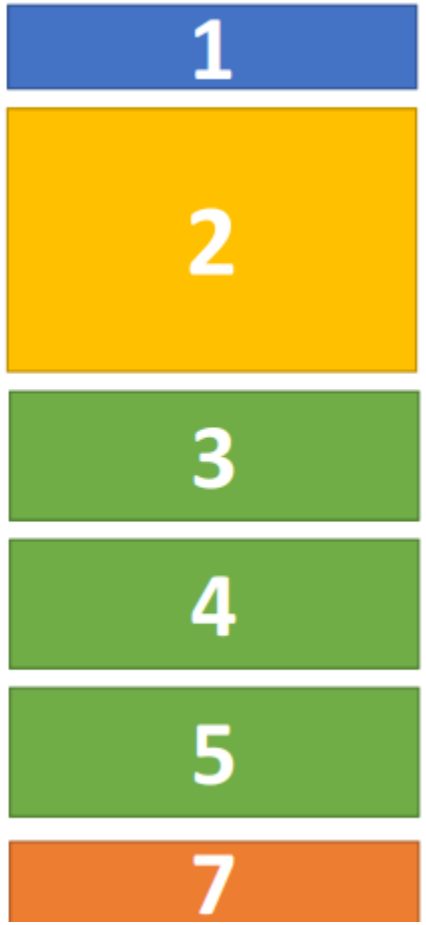
➜ Au contraire, possibilité de faire apparaître de nouveaux blocs et outils de navigation sur mobile comme le « menu burger »

LE RESPONSIVE DESIGN

Organisation classique
« sur ordinateur »



Organisation réduire
« sur mobile »



LES POINTS DE RUPTURE

➜ Qu'est-ce que les points de rupture ?

Ils représentent le moment ou l'affichage d'un site responsif change en fonction de la largeur / hauteur de l'écran

LES POINTS DE RUPTURE

➜ La balise HTML

```
<meta name="viewport">
```

Ils représentent le moment ou l'affichage d'un site responsif change en fonction de la largeur / hauteur de l'écran

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

LES POINTS DE RUPTURE

➜ Définir un point de rupture en CSS

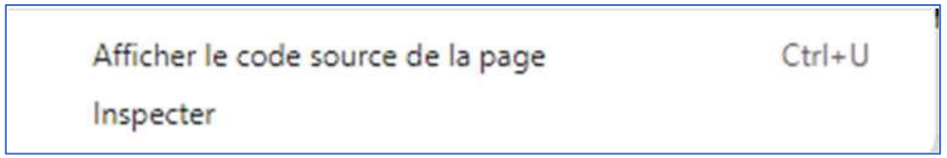
```
@media screen and (max-width: 480px) {

    div
    {
        display: block;
        width: 100%;
    }
}
```

La taille maximale (ou minimale) de l'écran doit être renseignée en paramètre via l'attribut `max-width` (ou `min-width`)

On pourra alors renseigner les modifications de style dans ce media

LES POINTS DE RUPTURE

➜ Simuler différentes tailles d'écran sur un navigateur



Clic-droit sur la page > Inspecter

ou F12 sur le clavier

Cliquer sur l'icone « Responsive »

A gauche sur Google Chrome et à droite sur Firefox

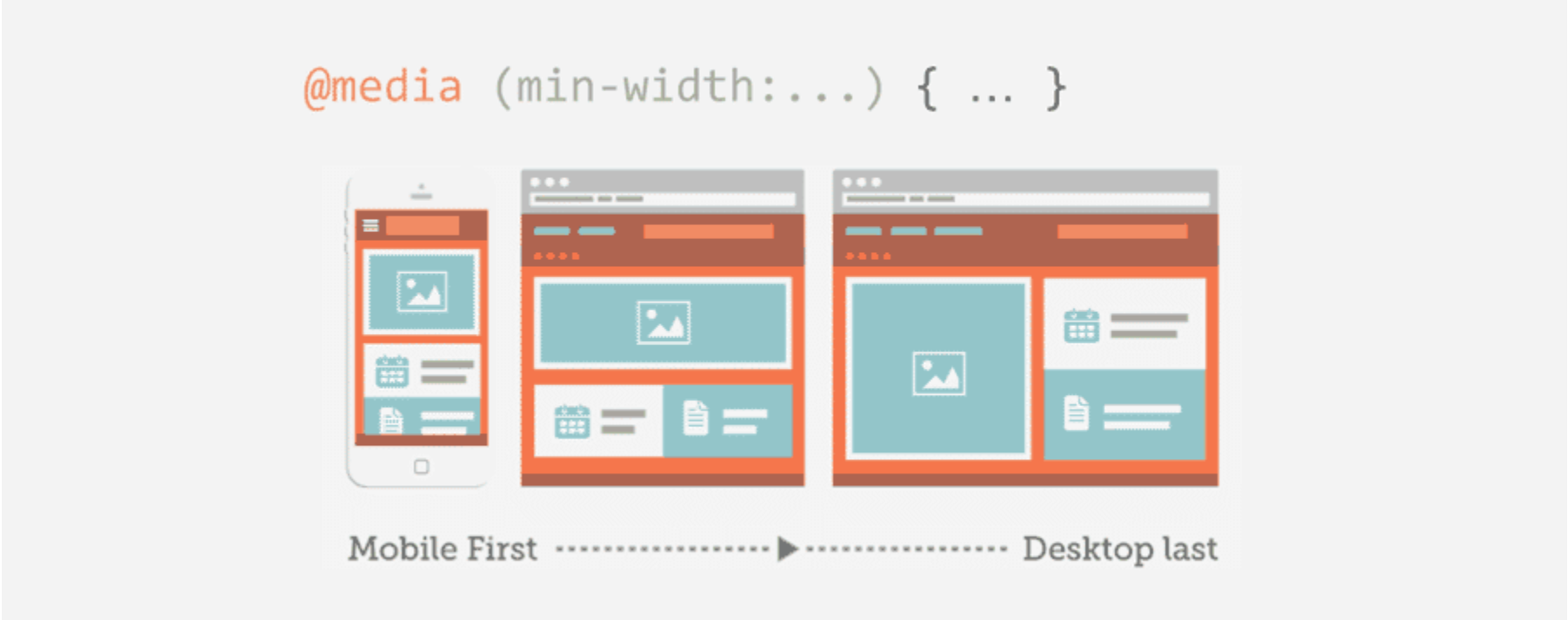
LES POINTS DE RUPTURE

➜ Les tailles d'écran communes et les valeurs des media-queries conventionnelles

Mobile :	360 x 640	576px
Mobile:	375 x 667	
Mobile :	360 x 720	
iPhone X :	375 x 812	
Pixel 2 :	411 x 731	
Tablette :	768 x 1024	768px
Ordinateur portable :	1366 x 768	992px
Ordinateur portable ou de bureau haute résolution :	1920 x 1080	1200px

LA BONNE PRATIQUE

➜ Faire du « Mobile First »



Design mobile-first (Source de l'image : silocreativo.com)

Le mode d'affichage

```
display: block;
```

- Permet de définir le mode d'affichage utilisé pour le rendu d'un élément et la disposition utilisée
- On y retrouve différentes valeurs possibles :

`block`	`grid`
`inline`	`flex`
`inline-block`	…

A VOUS DE JOUER !

Dans un nouveau document HTML avec un fichier de style associé :

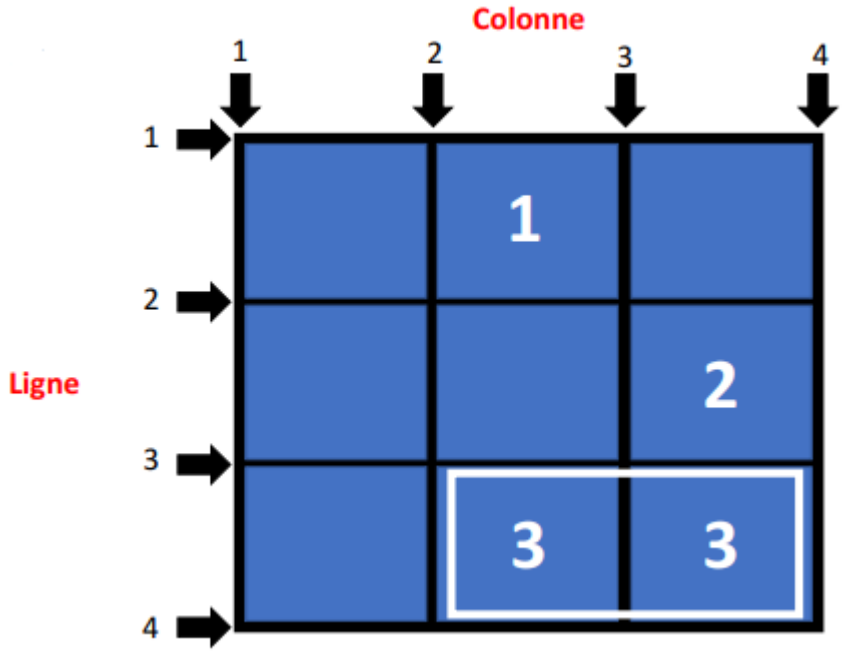
- Créer 3 blocs utilisant **un tier de la largeur** de la page en mode « ordinateur »
- Ces blocs seront, lorsque l'affichage basculera en « mobile », en **pleine largeur**

CORRECTION

```
/* Mobile First */
div.bloc {
    height: 10px;
    border: solid black 1px;

    width: 100%;
    display: block;
}

/* Point de rupture "ordinateur" */
@media screen and (min-width: 992px) {

    div.bloc {
        width: 30%;
        display: inline-block;
    }
}
```

LES GRIDS CSS

➜ **Grid** est une spécification CSS qui permet de créer des mises en page en **divisant** l'espace d'affichage



	Colonne	Ligne
1	Début : 2 Fin : 3	Début : 1 Fin : 2

	Colonne	Ligne
2	Début : 3 Fin : 4	Début : 2 Fin : 3

	Colonne	Ligne
3	Début : 2 Fin : 4	Début : 3 Fin : 4

LES PROPRIETES CSS

Le mode d'affichage

```
display: grid;
```

➜ Permet de définir le type d'affichage utilisé pour le rendu d'un élément et la disposition utilisée pour ses éléments enfants

Définition des colonnes

```
grid-template-columns: xxx;
```

➜ Permet de définir le nombre de colonnes ainsi que la largeur
➜ Cette propriété est à **ajouter au conteneur** et non aux éléments enfants
➜ Différentes valeurs :

`auto`	La largeur de la colonne s'adapte en fonction du conteneur
`50px`	La largeur de la colonne est fixée à 50px
`1fr`	Divise la largeur du conteneur en fraction
`25%`	La largeur est définie en pourcentage du conteneur
`...`	

LES PROPRIETES CSS

Définition des lignes

```
grid-template-rows: xxx;
```

➜ Permet de définir la configuration des lignes d'un conteneur
➜ Cette propriété est à **ajouter au conteneur** et non aux éléments enfants
➜ Différentes valeurs :

`auto`	La hauteur de la colonne s'adapte en fonction du conteneur
`50px`	La hauteur de la colonne est fixée à 50px
`1fr`	Divise la hauteur du conteneur en fraction
`25%`	La hauteur de la ligne est définie en pourcentage du conteneur
`...`	

LES PROPRIETES CSS

Définition des espacements

```
~~grid-~~gap: 5px;
```

- Permet de définir l'espace entre les lignes et les colonnes (entre les éléments enfants) dans la grille
- Cette propriété est à **ajouter au conteneur** et non aux éléments enfants
- La valeur peut être en unité pixels (ou `em`) ou en pourcentages.
- Ancienne propriété : « `grid-gap` », obsolète et remplacée par « `gap` » seule

LES PROPRIETES CSS

Définition de la position

```
grid-column: 1 / 2
```

➜ Permet de définir la colonne de début et de fin
➜ Cette propriété est à **ajouter aux éléments enfants** du conteneur

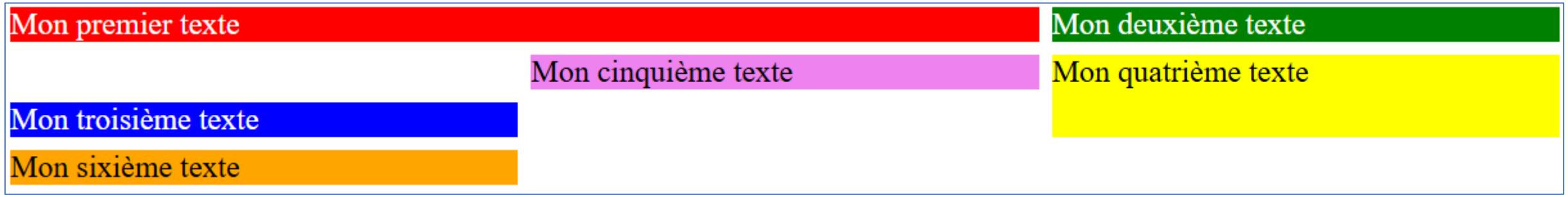
```
grid-row: 2 / 4
```

➜ Permet de définir la ligne de début et de fin
➜ Cette propriété est à **ajouter aux éléments enfants** du conteneur

A VOUS DE JOUER ! (1)

Reproduire l'affichage suivant :

```
<div class="container">
    <div class="one">Mon premier texte</div>
    <div class="two">Mon deuxième texte</div>
    <div class="three">Mon troisième texte</div>
    <div class="four">Mon quatrième texte</div>
    <div class="five">Mon cinquième texte</div>
    <div class="six">Mon sixième texte</div>
</div>
```



CORRECTION

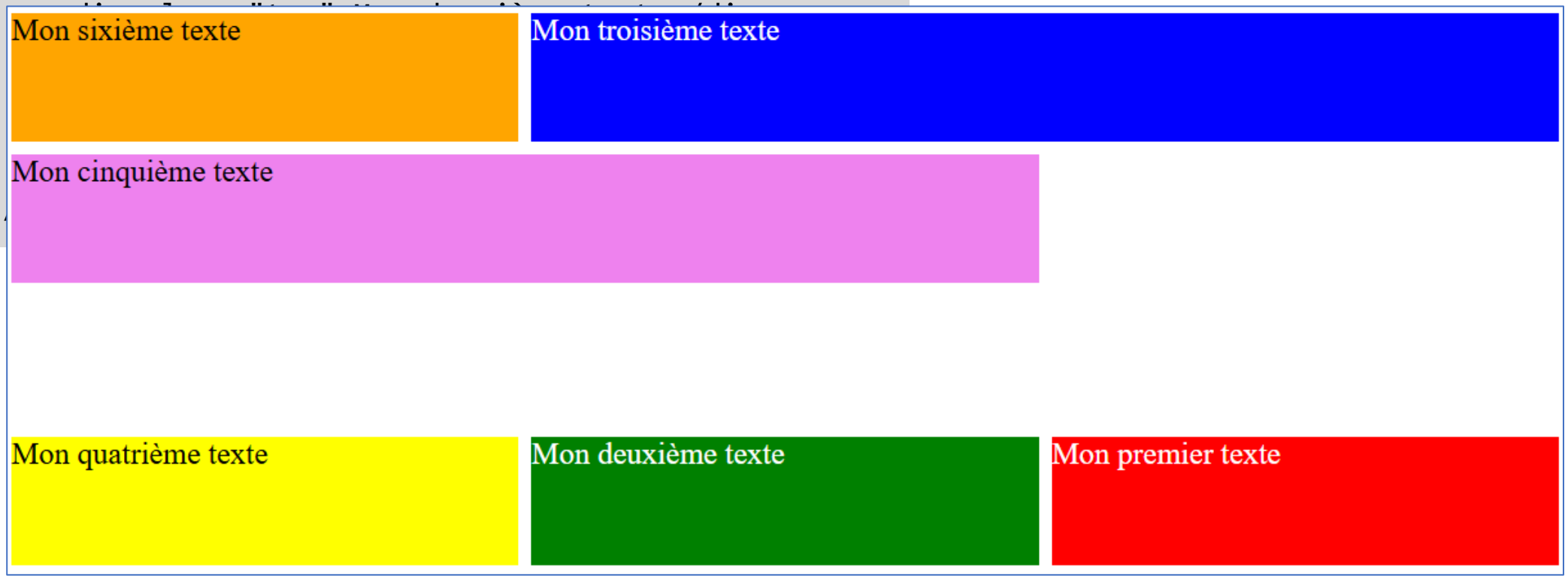
```
div.container {
    display: grid;
    gap: 10px;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: repeat(2fr 1fr);
}

div.one {
    background-color: red;
    color: white;
    grid-column: 1 / 3;
    grid-row: 1;
}

div.two {
    background-color: green;
    color: white;
    grid-column: 3 / 4;
    grid-row: 1 / 2;
}

div.three {
    background-color: blue;
    color: white;
    grid-column: 1;
    grid-row: 3 / 4;
}
```

```
div.four {
    background-color: yellow;
    grid-column: 3 / 4;
    grid-row: 2 / 4;
}

div.five {
    background-color: violet;
    grid-column: 2;
    grid-row: 2;
}

div.six {
    background-color: orange;
    grid-column: 1;
    grid-row: 4;
}
```

A VOUS DE JOUER ! (2)

Reproduire l'affichage suivant :

```
<div class="container">
    <div class="one">Mon premier texte</div>
```



GRID CSS

CORRECTION

```
div.container {
    display: grid;
    gap: 10px;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: repeat(4, 100px);
}

div.one {
    background-color: red;
    color: white;
    grid-column: 3 / 4;
    grid-row: 4;
}

div.two {
    background-color: green;
    color: white;
    grid-column: 2 / 3;
    grid-row: 4;
}

div.three {
    background-color: blue;
    color: white;
    grid-column: 2 / 4;
    grid-row: 1;
}
```

```
div.four {
    background-color: yellow;
    grid-column: 1;
    grid-row: 4;
}

div.five {
    background-color: violet;
    grid-column: 1 / 3;
    grid-row: 2;
}

div.six {
    background-color: orange;
    grid-column: 1;
    grid-row: 1;
}
```

A VOUS DE JOUER ! (3)

Reproduire l'affichage suivant :

GRID CSS

CORRECTION

```
div.container {
    display: grid;
    gap: 10px;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: 100px 200px 100px;
}

div.one {
    background-color: red;
    color: white;
    grid-column: 1 / 4;
    grid-row: 1;
}

div.two {
    background-color: green;
    color: white;
    grid-column: 3 / 4;
    grid-row: 3;
}

div.three {
    background-color: blue;
    color: white;
    grid-column: 1 / 2;
    grid-row: 2;
}
```

```
div.four {
    background-color: yellow;
    grid-column: 1 / 2;
    grid-row: 3;
}

div.five {
    background-color: violet;
    grid-column: 2 / 4;
    grid-row: 2;
}

div.six {
    background-color: orange;
    grid-column: 2 / 3;
    grid-row: 3;
}
```

LES FLEX BOX

➜ **Flex** est une spécification CSS permettant de définir la capacité d'un élément à remplir l'espace disponible dans son conteneur

Le mode d'affichage

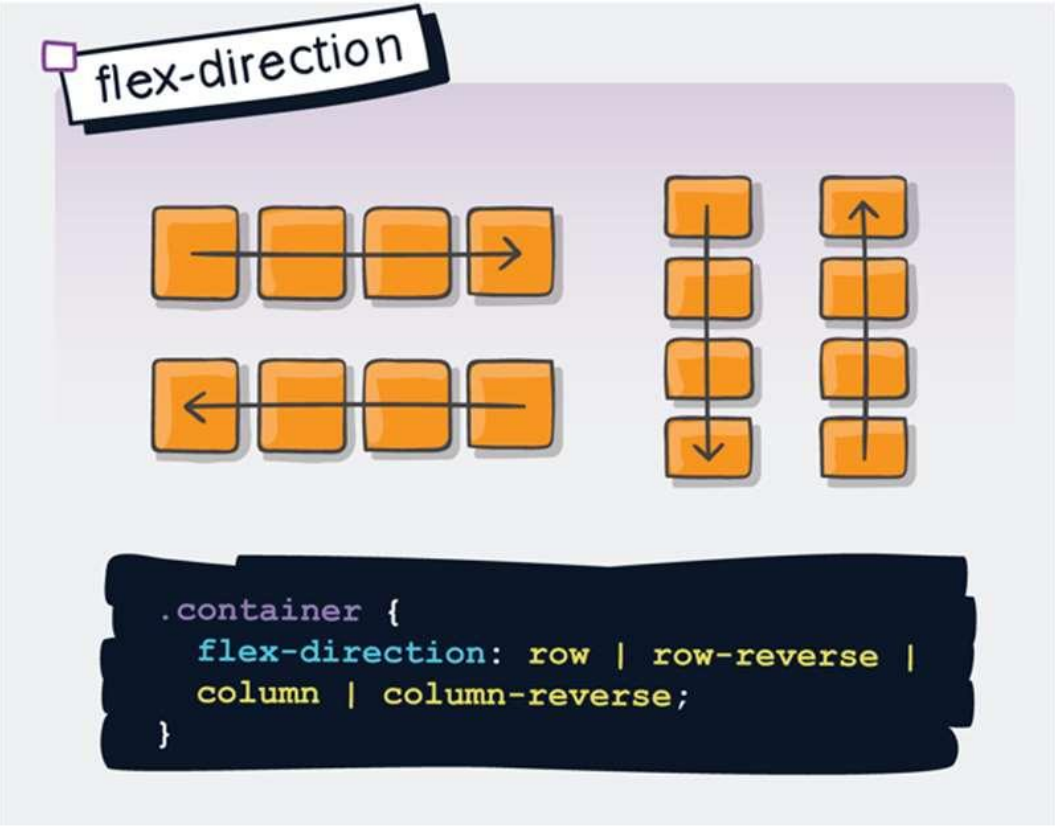
```
display: flex;
```

➜ Permet de définir le type d'affichage utilisé pour le rendu d'un élément et la disposition utilisée pour ses éléments enfants
➜ Cette propriété est à **ajouter au conteneur** et non aux éléments enfants

LES PROPRIETES CSS

Direction des éléments enfants

```
flex-direction: row;
```



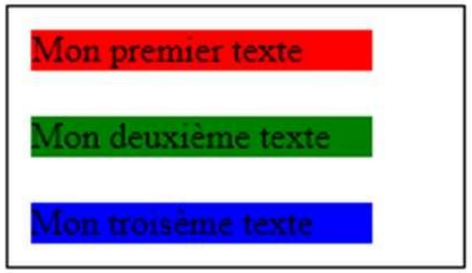
- ➜ Permet de définir la direction de l'axe que d
- ➜ Cette propriété est à **ajouter au conteneur** e
- ➜ Différentes valeurs :

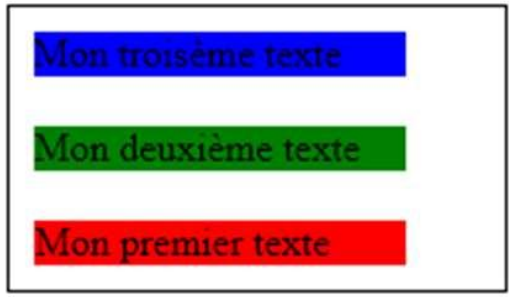
`row`	Direction en ligne
`column`	Direction en colonne
`row-reverse`	Direction en ligne, m
`column-reverse`	Direction en colonne, mais commence du côté opposé

A VOUS DE JOUER !

Reproduire les affichages ci-dessous :

```
<div class="container">
    <div class="item">Mon premier texte</div>
    <div class="item">Mon deuxième texte</div>
    <div class="item">Mon troisième texte</div>
</div>
```





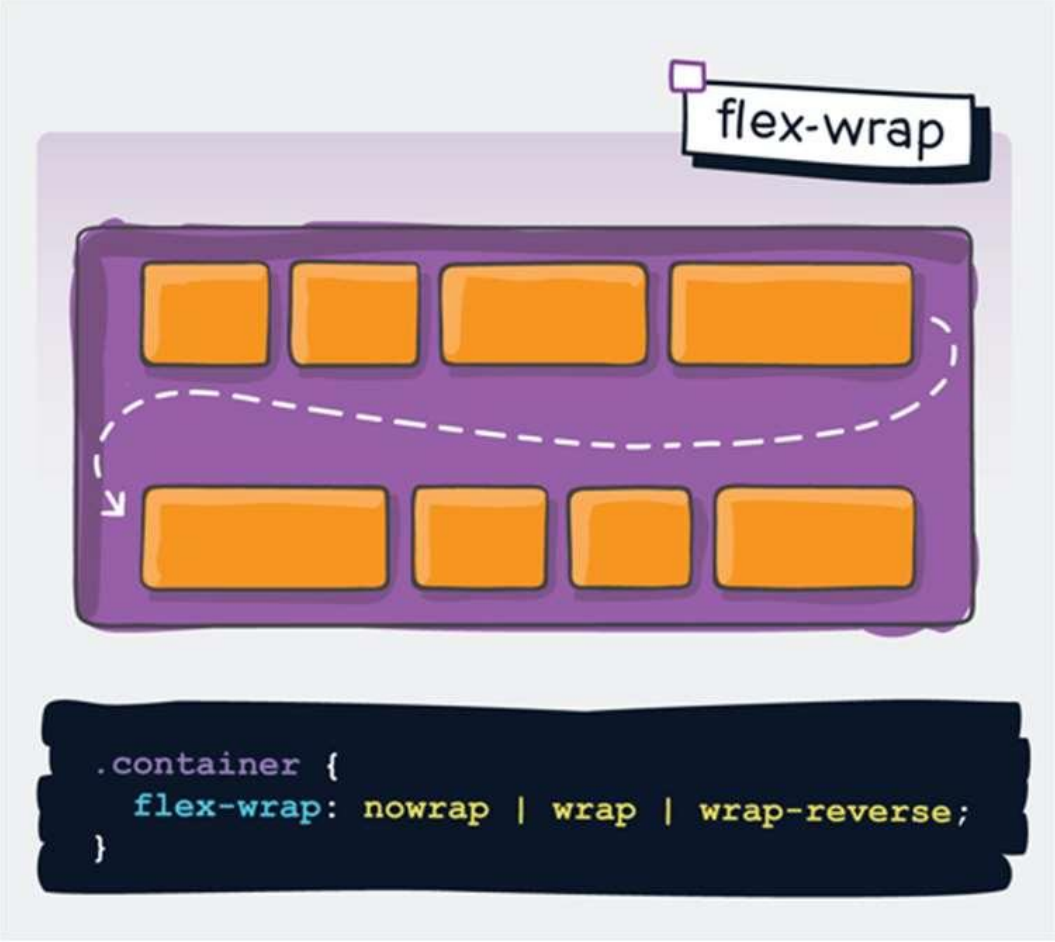
LES PROPRIETES CSS

Retours à la ligne

```
flex-wrap: wrap;
```

- Permet de définir si les éléments enfants d
  plusieurs lignes avec un retour à la ligne au
- Cette propriété est à **ajouter au conteneur**
- Différentes valeurs :

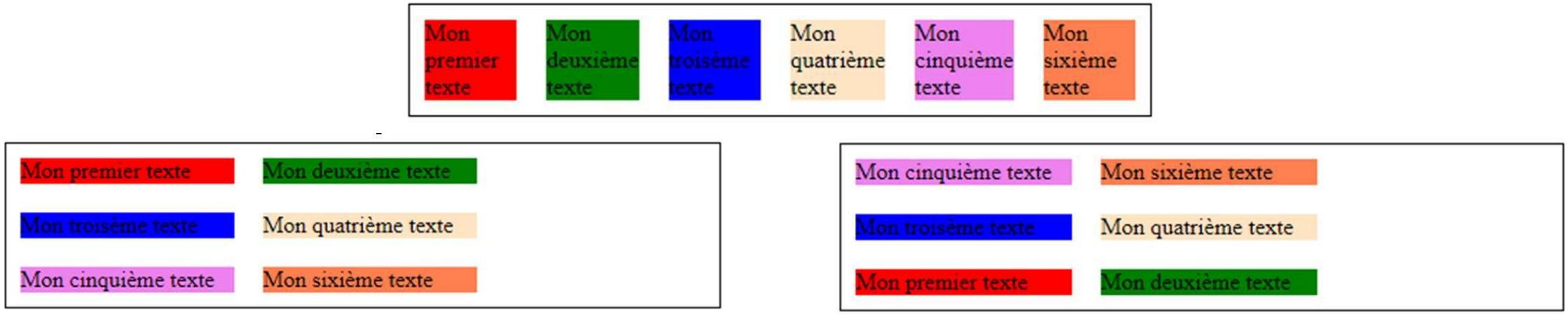
`nowrap`	L'ensemble des élér
`wrap`	Sur plusieurs lignes
`wrap-reverse`	Sur plusieurs lignes



A VOUS DE JOUER !

Reproduire les affichages ci-dessous :

```
<div class="container">
    <div class="one">Mon premier texte</div>
    <div class="two">Mon deuxième texte</div>
    <div class="three">Mon troisième texte</div>
    <div class="four">Mon quatrième texte</div>
    <div class="five">Mon cinquième texte</div>
    <div class="six">Mon sixième texte</div>
</div>
```



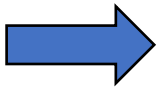
LES PROPRIETES CSS

Direction et retour à la ligne

```
flex-flow: row wrap;
```

➜ Permet de réunir les propriétés de direction et de retour à la ligne
➜ Cette propriété est à **ajouter au conteneur** et non aux éléments enfants

```
flex-direction: row;
flex-wrap: wrap;
```



```
flex-flow: row wrap;
```

LES PROPRIETES CSS

Occupation du contenu

```
justify-content: flex-start;
```

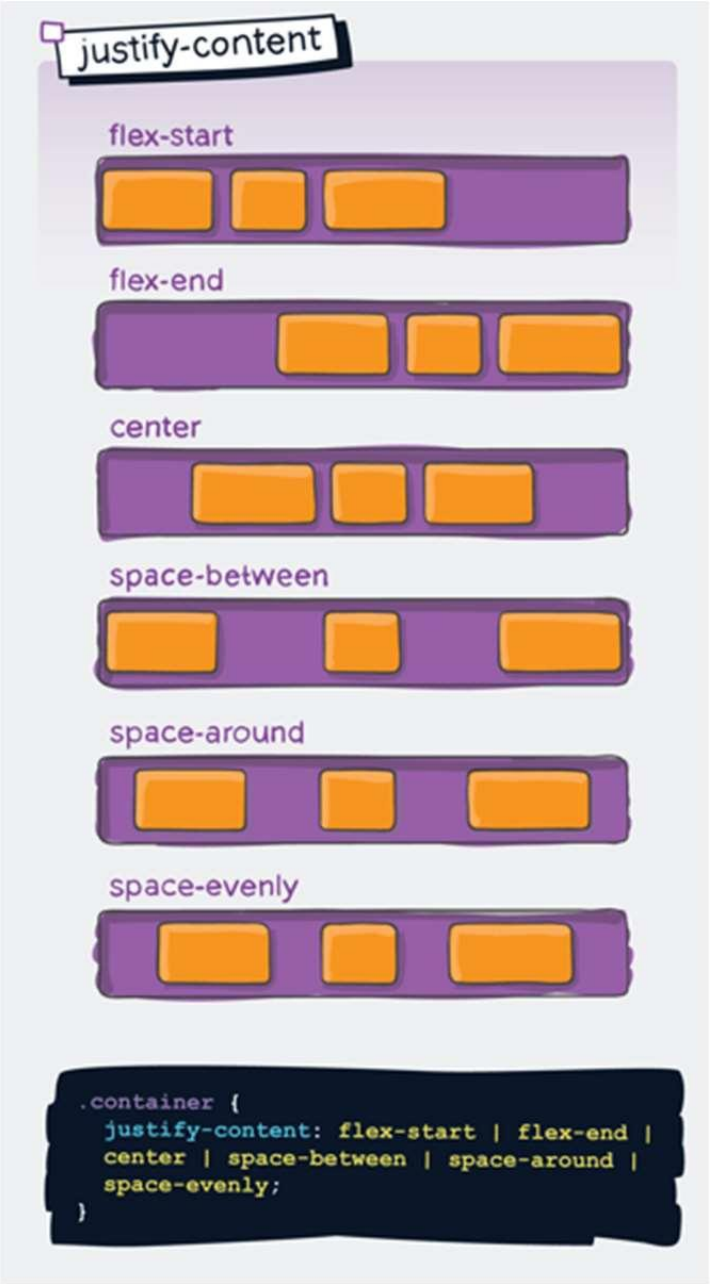
- Permet de définir commer vent occuper l'espace linéaire disponible
- Cette propriété est à ajou x éléments enfants

```
flex-start
flex-end
center
space-between
space-evenly
...
```

re

disponible

tablement



LES PROPRIETES CSS

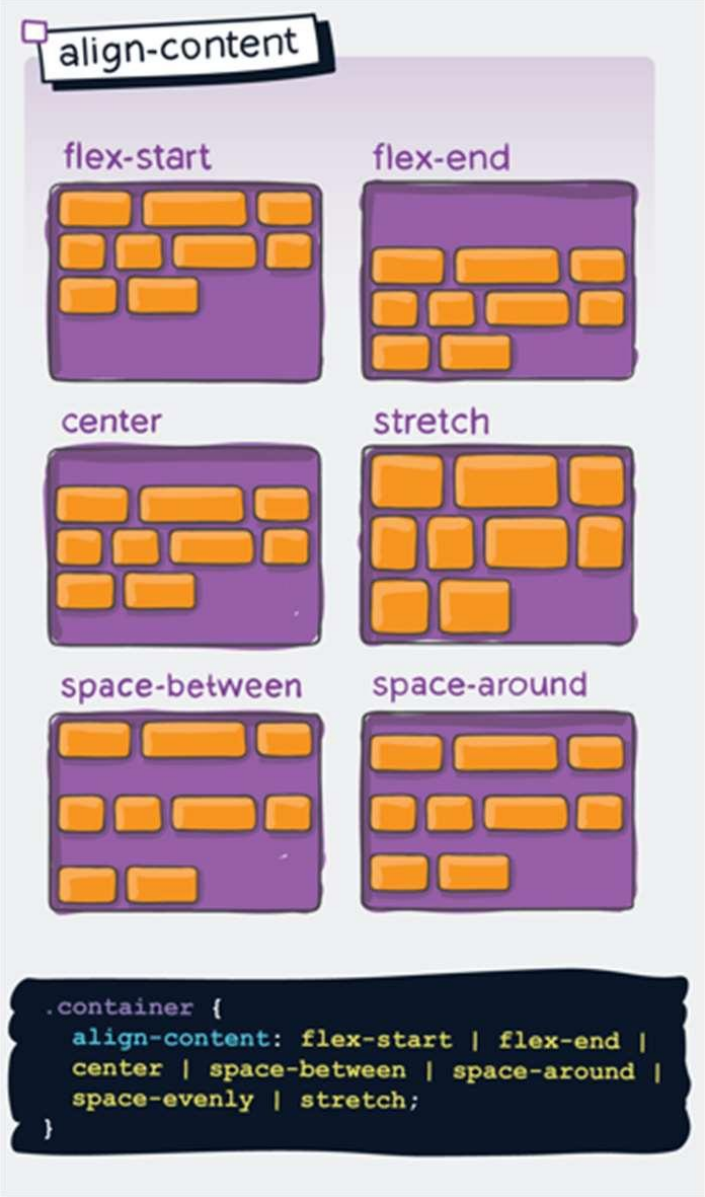
Alignement du contenu

```
align-content: flex-start;
```

➜ Permet de définir l'espace ce secondaire
➜ Cette propriété est à ajou ux éléments enfants

```
flex-start
flex-end
center
space-between
...
```

e disponible (du haut vers le bas)
enfants



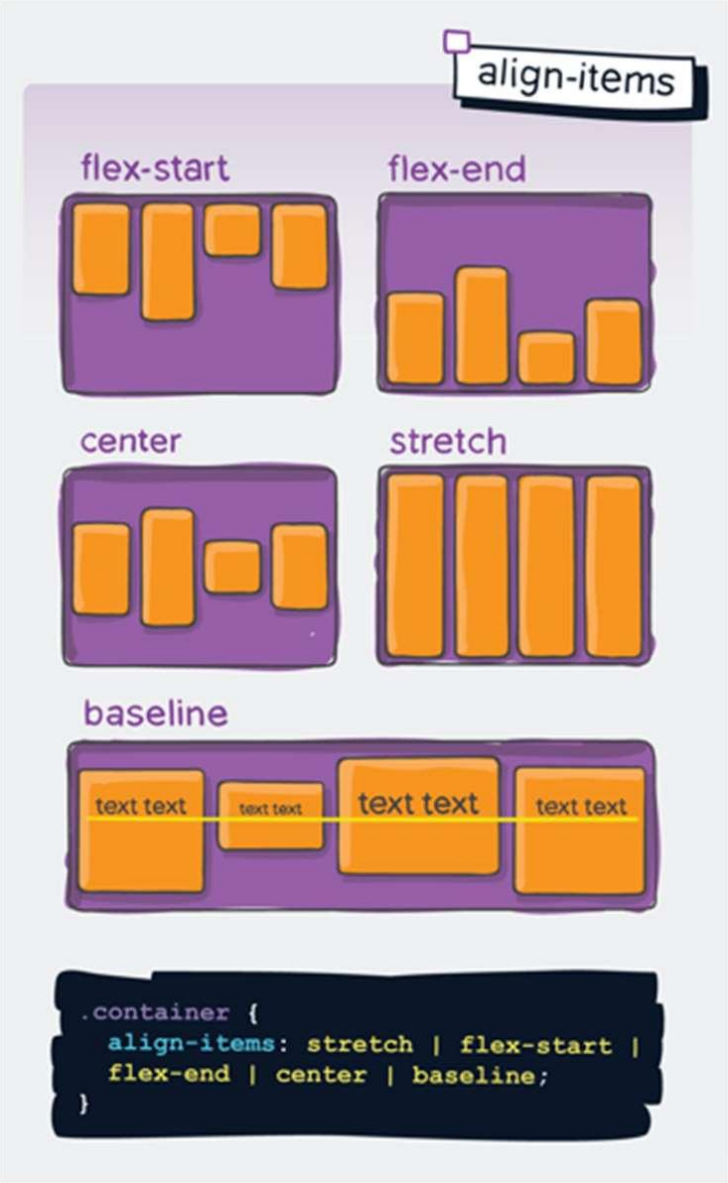
LES PROPRIETES CSS

Alignement du contenu

```
align-items: flex-start;
```

➜ Permet de définir l'aligne

➜ Cette propriété est à ajou ux éléments enfants



`flex-start`

`flex-end`

`center`

`stretch` disponible (de haut en bas) en nts

`baseline` centre en fonction du contenu

LES PROPRIETES CSS

Taille des éléments enfants

```
flex-grow: xxx;
```

- Permet de définir la taille que doit prendre l'élément enfant en cas d'agrandissement en fonction de la taille du conteneur
- Cette propriété est à **ajouter à l'élément enfant**
- **La valeur est numérique (sans unité) et divise la place disponible en fonction des enfants**

LES PROPRIETES CSS

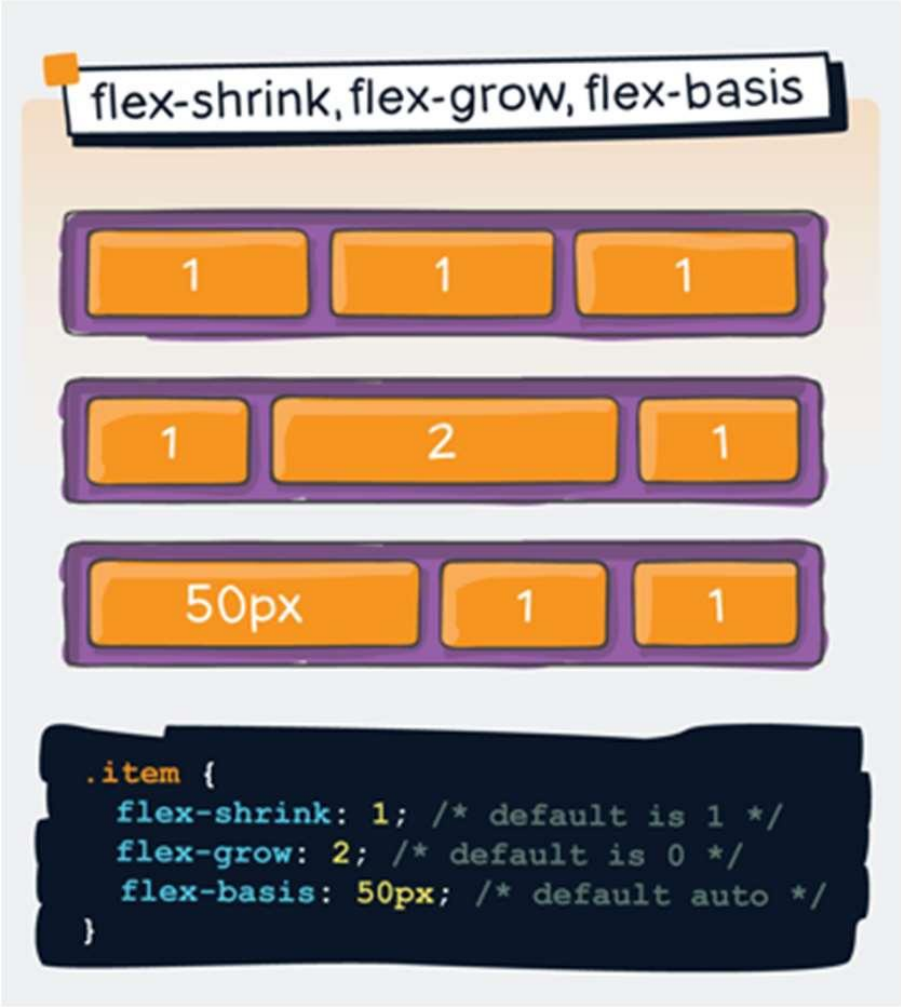
Taille des éléments enfants

```
flex-shrink: xxx;
```

- Permet de définir la taille que doit prendre l'élément enfant en cas de réduction de la taille du conteneur
- Cette propriété est à **ajouter à l'élément enfant**
- **La valeur est numérique (sans unité) et divise la place disponible en fonction des enfants**

LES PROPRIETES CSS

Taille des éléments enfants

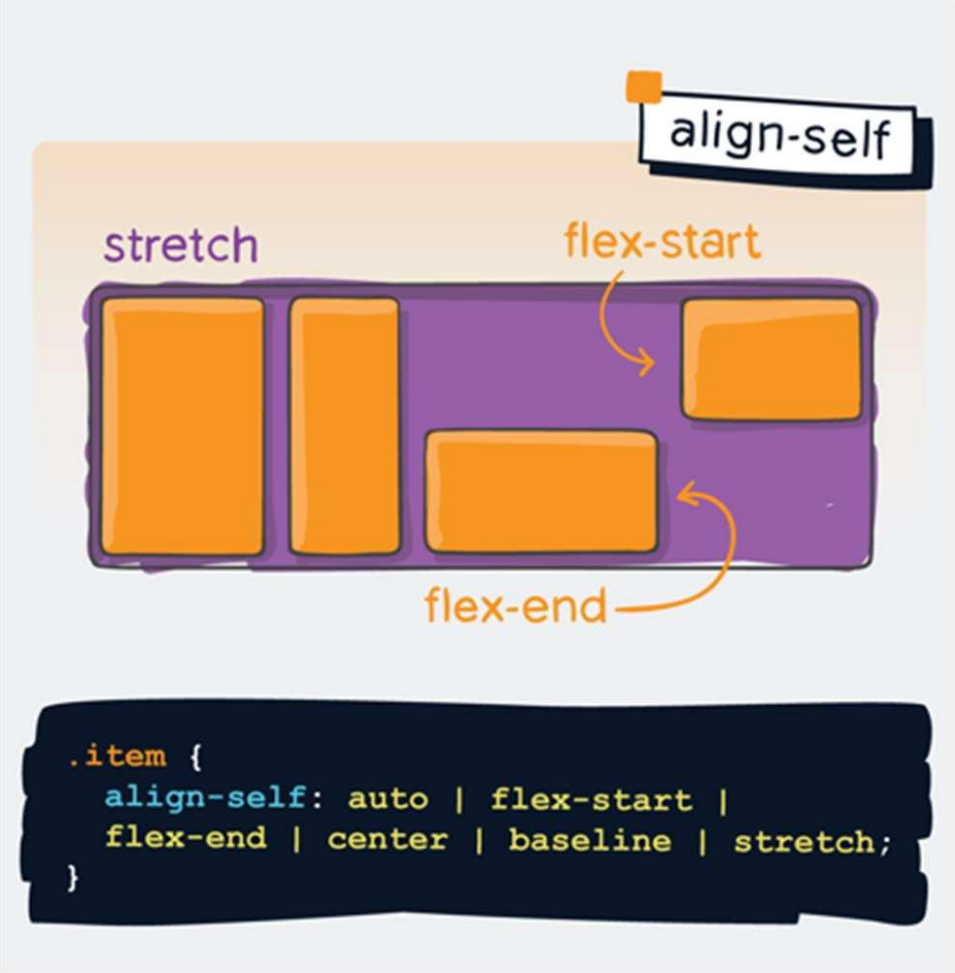


LES PROPRIETES CSS

Alignement des éléments

```
align-self: xxx;
```

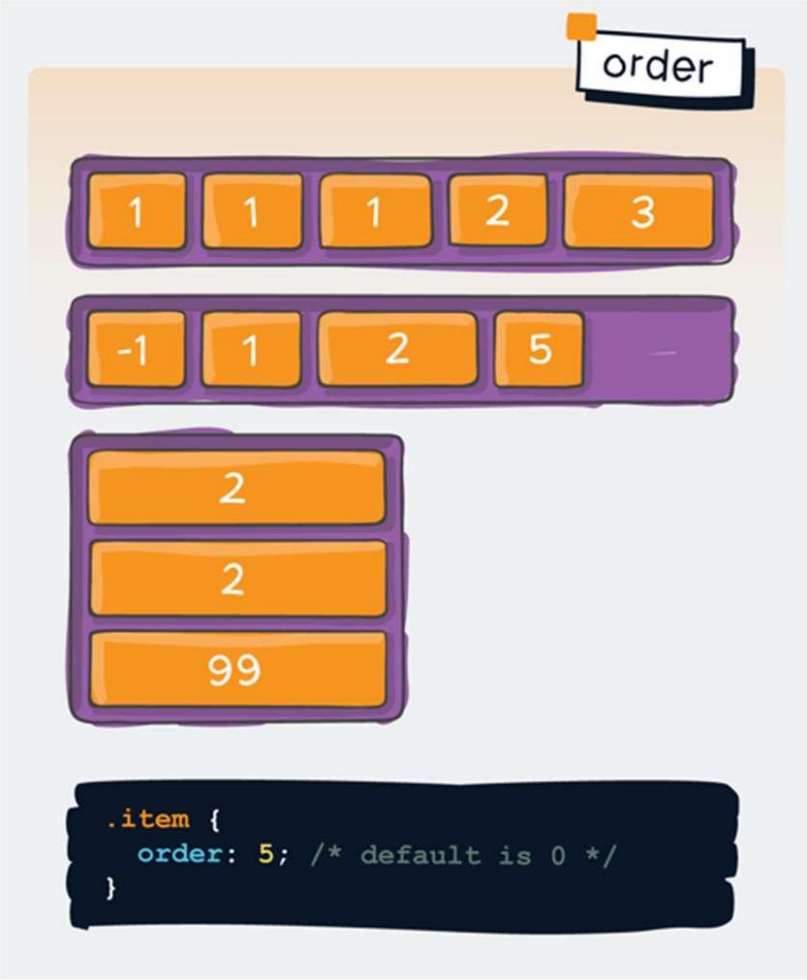
- Permet de définir l'alignement par défaut, e
  conteneur
- Cette propriété est à **ajouter à l'élément ent**



Ordre d'apparition

```
order: xxx;
```

➜ Permet de définir l'ordre d'apparition dans le flux
➜ Cette propriété est à **ajouter à l'élément enfant**



➜ Quelques liens utiles pour les Flex

https://css-tricks.com/snippets/css/a-guide-to-flexbox/

https://flexboxfroggy.com/

https://www.creativejuiz.fr/blog/css-css3/apprendre-positionnement-flexbox-s-amusant

ET LESS, SASS, SCSS…

- LESS : créé en 2009
- Très proche du CSS pour la syntaxe et compilé en JavaScript
- Son heure de gloire s'est produite avec Bootstrap 3 !
- Fonctionnalités de variables, d'imbrication de blocs, des fonctions basiques…

```
@primary-color: #4a90e2;
@padding-base: 16px;

.button {
    color: @primary-color;
    padding: @padding-base;

    &:hover {
      color: darken(@primary-color, 10%);
    }
}
```

ET LESS, SASS, SCSS…

- SASS : créé en 2006
- Le préprocesseur CSS le plus ancien
- Syntaxe basée sur les indentations ! Sans accolades, ni points-virgules

```
$primary-color: #4a90e2
$padding-base: 16px

.button
    color: $primary-color
    padding: $padding-base

    &:hover
        color: darken($primary-color, 10%)
```

ET LESS, SASS, SCSS…

- SCSS : créé en 2010
- C'est du SASS avec une syntaxe CSS
- Tout CSS valide est du SCSS valide
- Aujourd'hui, lorsque l'on dit « SASS », on parle presque toujours de SCSS

```
$primary-color: #4a90e2;
$padding-base: 16px;

.button {
    color: $primary-color;
    padding: $padding-base;

    &:hover {
        color: darken($primary-color, 10%);
    }
}
```

LES VARIABLES EN CSS

➜ **Depuis 2017**, les variables sont disponibles en CSS natif !

```
:root {
    --primary-color: #4a90e2;
    --padding-base: 16px;
}

.button {
    color: var(--primary-color);
    padding: var(--padding-base);
}
```

➜ **En CSS natif, les variables sont dynamiques** : modifiables en JavaScript. Ce qui n'est pas le cas en SASS/SCSS car elles sont compilées

```
elButton.style.setProperty('--primary-color', '#ff0000');
```

LES VARIABLES EN CSS

➜ Possible de « surcharger » des variables :

```
:root {
    --bg-color: #4a90e2;
}

.button {
    background-color: var(--bg-color);
}

.button-success {
    --bg-color: #38ED31;
}
```

```
<a href="#" class="button">Bouton 1</a>
```

Bouton 1

```
<a href="#"
   class="button button-sucess">Bouton 1
</a>
```

Button 2

➜ On pourra plus facilement regrouper les propriétés en commun et par défaut dans un sélecteur et ne spécifier que les changements de variables dans les sélecteurs spécifiques

SELECTEUR IMBRIQUÉ

➜ Depuis 2023, et quasi-universel dans les navigateurs depuis 2024

Chrome	Edge *	Safari	Firefox	Opera	IE
4-108	12-108			10-94	
[1] 109-111	[1] 109-111	3.1-16.4	2-114	[1] 95-97	
[3] 112-119	[3] 112-119	[3] 16.5-17.1	[2] 115-116	[3] 98-105	
120-145	120-144	17.2-26.2	117-147	106-126	6-10
146	145	26.3	148	127	11
147-149		26.4-TP	149-151		

Chrome for Android	Safari on iOS *	Samsung Internet	Opera Mini *	Opera Mobile	Browser for Android	Android Browser *	Firefox for Android	QQ Browser	Baidu Browser	KaiOS Browser
	3.2-16.4	4-22								
	[3] 16.5-17.1	[3] 23-24								
	17.2-26.2	25-28		12-12.1		2.1-4.4.4				2.5
145	26.3	29	all	80	15.5	145	147	14.9	13.52	3.1
	26.4									

https://caniuse.com/css-nesting

SELECTEUR IMBRIQUÉ

➜ Depuis 2023, et quasi-universel dans les navigateurs depuis 2024

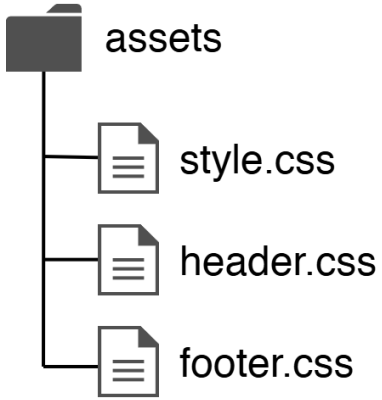
```
.button {
    color: blue;
    padding: 16px;

    &:hover {
        color: darkblue;
    }

    .icon {
      margin-right: 8px;
    }
}
```

IMPORTER DES FEUILLES DE STYLES

➜ CSS possède **@import** permettant d'importer des fichiers CSS secondaires

style.css

```
@import "header.css";
@import "footer.css";
```



➜ A l'heure actuelle, @import crée des requêtes HTTP séparées pour chaque fichier alors que SCSS, du fait de la compilation, ne génère pas de trafic supplémentaire